CS 556

Logistic Regression

What to submit:

1. You will be provided with a Jupyter notebook to give your requisite code / analysis.

2. In addition, you can use this word document to answer all the non-coding questions (inline) and submit a single document for all the analysis. (Questions will mostly be localized to the "Discussion" section of each question.

Submission format: You need to submit a zipped archive containing the jupyter notebook with your code and a pdf file named *questions_and_solutions.pdf* (i.e., the current file with the solutions that you will fill in and convert to PDF format).

Each of your Jupyter notebooks should be directly executable without any further modifications (i.e., if your assignment submission is downloaded from canvas and unzipped), any of your submitted notebooks if executed using a Jupyter server should be directly runnable without additional data downloads etc.

Titanic Classification Problem

a. **Dataset Description:** The sinking of the Titanic is one of the most infamous shipwrecks in history. On April 15, 1912, during her maiden voyage, the widely considered "unsinkable" RMS Titanic sank after colliding with an iceberg. Unfortunately, there weren't enough lifeboats for everyone onboard, resulting in the death of 1502 out of 2224 passengers and crew. While there was some element of luck involved in surviving, it seems some groups of people were more likely to survive than others. Our goal is to develop a classifier to predict whether a passenger *survived* the calamity. The attributes in the datasets are as follows:

   i. PassengerID (categorical): Passenger ID

   ii. Ticket (categorical): Ticket Number

   iii. Name (categorical): Passenger Name

   iv. Cabin (categorical): Passenger Cabin

   v. Pclass (categorical): Passenger Class (1 = 1st; 2 = 2nd; 3 = 3rd)

   vi. Sex (categorical): Passenger Sex

   vii. Age (continuous): Passenger Age

   viii. SibSp (continuous): Number of Siblings/Spouses Aboard

   ix. Parch (continuous): Number of Parents/Children Aboard

   x. Fare (continuous): Passenger Fare

   xi. Embarked (categorical): Port of Embarkation (C = Cherbourg; Q = Queenstown; S = Southampton)

xii. Survived (categorical): Survival (0 = No;1 = Yes) --- this is the target column. Make sure to encode data appropriately for each classifier.

Our target variable will be ***Survived*****.** Use the rest of the fields mentioned above to predict whether a passenger survived the Titanic shipwreck.

b. Data Loading / Preprocessing

i. Loading

1. Load the data <df_train.csv> and <df_test.csv> as a pandas dataframe using the `pandas.read_csv` function. The 'df_test.csv' has been preprocessed (i.e., null values have been dropped, certain columns etc. have been dropped) and should not be changed apart from splitting the dataframe into X_test and y_test. The 'df_train' data has NOT been preprocessed and you will need to preprocess and prepare the 'df_train' dataframe. Note: Neither df_train nor df_test have been scaled. The next few steps will enumerate data preprocessing, scaling requirements we need to perform.

2. The resulting dataframe (i.e., df_train) should have the shape (712,12) indicating that there are 712 instances and 12 columns.

3. In df_train dataframe, currently you have 12 columns which are the following – PassengerID, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked and the Survived column (target variable).

4. Use the `pandas.isnull().sum()` function to check if there are any missing values in the df_train dataframe. Report which columns have missing (i.e., null) values and provide the number of the null values in the columns.

5. Use the `pandas.DataFrame.drop()` function to drop the 'Cabin', 'PassengerID', 'Name' and 'Ticket' columns.

6. Use the `pandas.DataFrame.fillna()` function to replace the NA values in the 'Age' column with the mean value of the 'Age'

column. Note: This process is called **imputation** (i.e., filling null values with a pre-specified value) and we are employing one strategy called mean imputation, but other strategies can also be employed in general. Use the `dropna()` function to drop any remaining **rows** that consist of NA values..

7. Your task is to use the feature columns to predict the target column. This can be cast as a classification problem.

8. Create a pandas dataframe X_train of features (by dropping the 'Survival' column from the df_train dataframe). Create a pandas Series object of targets y_train (by only considering the 'Survival' column from the df_train dataframe). Moving forward, we will be working with X_train and y_train. At this point, also split the df_test into X_test and y_test by dropping the 'Survival' column and storing the features into X_test. Store the 'Survival' column in y_test.

ii. Data Visualization

1. Employ a scatter plot using `matplotlib.pyplot.scatter` between the age of the passengers and the price of their fare. Label the x-axis and the y-axis and also give the plot a title.
    a. What is the highest and lowest fare price?
    Ans: Highest Fare: 512.33

    Lowest Fare: 0.00

    b. What are the respective mean values of these two features?
    Ans: Mean Age: ~29.70 years

    Mean Fare: ~32.20 units

    c. What was the age of the oldest passenger?
    Ans: Oldest Passenger Age: 80.0 years
2. Use the df_train dataframe. Using matplotlib visualize the number of males and females that survived and their respective passenger

classes on two separate bar chart plots using `matplotlib.pyplot.bar` (Passenger Class column)

a. Which *class of passengers* had the least number of survived males and how many? Repeat this analysis for females.
Ans: Least male survivors were in 3rd Class with 47 survivors
Least female survivors were in 2nd Class with 70 survivors

b. Which *class of passengers* had the greatest number of survived males and how many? Repeat this for females.
Ans: Most male survivors were in 1st Class with 45 survivors
Most female survivors were in 3rd Class with 72 survivors

3. Using the Target variable (Survived) in y_train plot a bar chart showing the distribution of the 'Survived' column.

a. What initial comment can you make about this distribution in terms of the frequency of each class?
Ans: 437 passengers (about 61%) did not survive

275 passengers (about 39%) survived

4. So far you should have successfully been able to load, preprocess and visualize your data. Use the `pd.get_dummies()` function to convert categorical data into dummy variables ('Sex' and 'Embarked'). Make sure to pass `drop_first=True` to the `get_dummies()` function. (Perform this only on X_train and store the result back into X_train).

a. What is the new shape of X?

Ans: The new shape of X_train is (712, 14)

iii. Data Scaling

1. Use MinMaxScaler to scale only the **continuous attributes** of X_train. Apply the `fit_transform()` function of the scaler to obtain the scaled data and store it back in X_train.

2. Scale the X_test using the scaler you have just fit, this time using the `transform()` function. Note: Store the scaled values back into X_test. At the end of this step, you must have X_train, X_test, all scaled according to the MinMaxScaler.

c. Modelling

i. Modelling (Model Instantiation / Training) using **Logistic Regression classifier**

1. Employ the Logistic Regression classifier from sklearn and instantiate the model. Label this model as 'model_lr'

2. Once instantiated, `fit()` the model using the scaled X_train, y_train data.

3. Employ the `predict()` function to obtain predictions on X_test and store this in a variable labeled as 'y_pred_lr'.

4. Employ the `accuracy_score` function by using the 'y_pred_lr' and 'y_test' variables as the functions parameters and print the accuracy of the Logistic Regression model.

ii. **Modelling Logistic Regression Classifier** with the addition of noise on the target variable.

In the data repository you should see three noisy datasets – `df_train_noise20`, `df_train_noise40`, and `df_train_noise60`. These datasets have already been preprocessed. In each dataset `df_train_noise<integer>`, the integer indicates the percentage of noise injected into the target variable in that training set. The noise can be considered a result of incorrect class labelling of a particular instance. For example, in `df_train_noise20`, 20% of the instances have an incorrect target label in the training set. Our goal will be to train a set of classification models on such noisy training data and test on a clean test set (i.e., same as what we have been using so far `df_test`).

Load the `df_train_noise<nl>` (nl means noise level and is a place holder for the integer percentage) datasets and split the data into `X_train_<nl>` and `y_train_<nl>` (e.g., if working with `df_train_noise20` we would split the data and store it in variables named `X_train_20`, `y_train_20`). `X_train_<nl>` should store the features and `y_train_<nl>` should store the target variable.

Repeat the following steps (c. i. 1 – 4) for the 20%, 40%, 60% noise level datasets.

Train a new Logistic Regression model on the new training and use the

pre-existing `X_test` and `y_test` to evaluate your model. Label this model as `model_lr_noise_<nl>`. Specifically, do the following:

1. Employ a new Logistic Regression classifier from sklearn and instantiate the model. Label this model as 'model_lr_noise<nl>'

2. Once instantiated, `fit()` the model using the X_train_<nl> and y_train_<nl> data.

3. Employ the `predict()` function to obtain predictions on X_test and store this in a variable labeled as 'y_pred_lr_noise<nl>'.

4. Employ the 'accuracy_score' function and print the accuracy of the new Logistic Regression model.
    a. What is your initial observation of the accuracy, is the accuracy higher or lower than that of the clean dataset?

d. Evaluation

i. Report F1 Score, Precision, Recall, Accuracy (All on the test set X_test, y_test)

1. Use `classification_report()` function from sklearn.metrics to report the precision, recall, and f1 score for each class for the `model_lr` model, along with a confusion matrix.

ii. Report the accuracy and classification report for each of the three noisy models (`model_lr_noise<nl>`)

iii. Make a calibration plot for 'model_lr'. Use the CalibrationDisplay class from sklearn.calibration to make your plot.

e. Discussion

i. Report the performance of the Logistic Regression (model_lr) classifier trained on the clean training data sets. Mention the specific numbers (i.e., from your results calculating the precision, recall, F1 score) in a table.

Model Name	Class	Precision	Recall	F1
model_lr	0	0.78	0.91	0.83

model_lr	1	0.78	0.59	0.67

ii. Report the performance of all the classification models you have trained thus far in terms of (precision, recall, F1 scores). From the results of the three noisy models implemented using Logistic Regression, what did you notice about the accuracy as the noise of the dataset increased?

Model Name	Class	Precision	Recall	F1
model_lr	Not Survived	0.78	0.91	0.83
model_lr	Survived	0.78	0.59	0.67
model_lr_noise20	Not Survived	0.75	0.85	0.79
model_lr_noise20	Survived	0.70	0.44	0.54
model_lr_noise40	Not Survived	0.72	0.80	0.76
model_lr_noise40	Survived	0.68	0.37	0.48
model_lr_noise60	Not Survived	0.70	0.75	0.72
model_lr_noise60	Survived	0.63	0.24	0.35